

Uso de Inteligencia Artificial (Agentes)

El corazón de la plataforma es su sistema multi-agente, diseñado para automatizar tareas que tradicionalmente requieren intervención humana experta. El sistema utiliza Modelos de Lenguaje Grande (LLMs) avanzados para razonar, generar contenido y evaluar código.

Nota Importante: El sistema es agnóstico al modelo específico y utiliza una capa de abstracción para comunicarse con proveedores de IA de alto rendimiento.

Agentes Implementados

El sistema orquesta tres agentes principales que interactúan en diferentes etapas del flujo:

1. Agente Orquestador (The Orchestrator)

- **Rol:** Gerente de Proyecto / Coordinador.
- **Ubicación en Código:** `handleToggleBotAssignment`, `adjudicateCandidateResult`.
- **Propósito:** Es el cerebro que toma decisiones de negocio. No genera código ni evalúa técnicamente en profundidad, sino que decide “qué hacer”.
- **Flujo de Trabajo:**
 1. Analiza el perfil del candidato entrante.
 2. Decide si existe una prueba adecuada en la biblioteca o si debe encargar una nueva al *Agente Arquitecto*.
 3. Al finalizar la prueba, recibe el reporte del *Agente Evaluador* y toma la decisión final de contratación (Aprobado/Rechazado) basándose en reglas de negocio predefinidas (ej. “Veto automático si hay fallos de seguridad”).

2. Agente Arquitecto (The Architect)

- **Rol:** Líder Técnico / Diseñador de Pruebas.
- **Ubicación en Código:** `analyzeCandidateProfile`, `generateAssessmentContent`.
- **Propósito:** Crear contenido técnico personalizado y de alta calidad.
- **Flujo de Trabajo:**
 1. Se activa cuando el Orquestador no encuentra una prueba compatible.
 2. Analiza el rol y seniority del candidato (ej. “Senior Java Developer”).
 3. Diseña una estructura de prueba (temas, dificultad).
 4. Genera preguntas específicas (código, teóricas, casos de uso) asegurando variedad y pertinencia técnica.
 5. Entrega la prueba estructurada en formato JSON lista para ser renderizada.

3. Agente Evaluador “Avalia” (The Evaluator)

- **Rol:** Entrevistador Técnico Senior.
- **Ubicación en Código:** `gradeSubmissionWithAI`.
- **Propósito:** Analizar en profundidad las respuestas del candidato.
- **Flujo de Trabajo:**
 1. Recibe el contexto de la prueba (preguntas correctas) y las respuestas del candidato (código, texto).
 2. Realiza un análisis estático del código enviado.
 3. Evalúa criterios múltiples:
 - **Corrección Funcional:** ¿El código hace lo que debe?
 - **Seguridad:** ¿Hay vulnerabilidades (SQLi, XSS)?
 - **Performance:** ¿Es eficiente (Big O notation)?
 - **Buenas Prácticas:** ¿El código es limpio y mantenible?
 4. Genera un reporte detallado con un puntaje (0-100) y una recomendación de contratación justificada.

Flujo de Interacción de Agentes

sequenceDiagram

```
participant C as Candidato
participant O as Agente Orquestador
participant A as Agente Arquitecto
participant E as Agente Evaluador (Avalia)
```

Note over O: Nuevo Candidato Detectado

O->>O: ¿Existe prueba compatible?

alt No existe prueba

O->>A: Solicitar diseño de prueba (Rol/Nivel)

A->>A: Generar preguntas y ejercicios

A-->>O: Prueba Nueva (JSON)

end

O->>C: Asignar y Enviar Prueba

C->>C: Realiza la Prueba

C->>E: Envía Respuestas

E->>E: Análisis de Código, Seguridad y Performance

E-->>O: Reporte de Evaluación (Score + Recomendación)

O->>O: Adjudicación Final (Reglas de Negocio)

O-->>C: Resultado Final (Aprobado/Rechazado)

Ingeniería de Prompts

El sistema utiliza técnicas avanzadas de Prompt Engineering: * **Role-Playing:** Se instruye a la IA para actuar como roles específicos (“Actúa como Arquitecto de Software...”). * **Chain of Thought:** Se pide a los agentes que razonen paso a paso antes de concluir. * **JSON Mode Enforcement:** Se fuerza a los modelos a responder estrictamente en formato JSON validado contra esquemas predefinidos, garantizando que la aplicación nunca falle por errores de parseo.